# 6주차 예습과제 (pp.308-362)

| | |
|---|---|
| 상태 | Done |
| 담당자 | 이서윤(인공지능대학 인공지능데이터사이언스학부) |
| 마감일 | 10/12/2026 |
| 작업 유형 | |

## 5. 회귀

### 01. 회귀 소개

- 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법
- 여러 개의 독립변수와 한 개의 종속변수 간의 상관관계를 모델링하는 기법을 통칭

$$\hat{y} = w_1 X_1 + w_2 X_2 + \cdots + w_p X_p + w_0$$

**ŷ**: 예측된 종속변숫값

**X**: 독립변수 (설명변수)

$w_1$ (또는 a, $\beta_1$): 회귀 계수(가중치) 및 직선의 기울기

- 머신러닝 회귀 예측의 핵심은 주어진 피처와 결정 값 데이터 기반에서 학습을 통해 최적의 회귀 계수를 찾아내는 것

| 독립변수 개수 | 회귀 계수의 결합 |
|---|---|
| 1개: 단일 회귀 | 선형: 선형 회귀 |
| 여러 개: 다중 회귀 | 비선형: 비선형 회귀 |

- 분류는 예측값이 카테고리와 같은 이산형 클래스 값이고 회귀는 **연속형 숫자 값**이다

**선형 회귀**

실제 값과 예측값의 차이(오류의 제곱 값)를 최소화하는 직선형 회귀선을 최적화하는 방식이며 선형 회귀 모델은 규제(Regularization) 방법에 따라 다시 별도의 유형으로 나뉠 수 있다

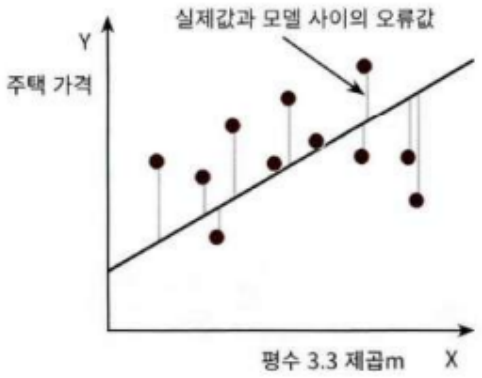
규제 - 일반적인 선형 회귀의 과적합 문제를 해결하기 위해서 회귀 계수에 페널티 값을 적용하는 것

## 02. 단순 선형 회귀를 통한 회귀 이해

**단순 선형 회귀** - 독립변수도 하나, 종속변수도 하나인 선형 회귀

$$\hat{y} = w_1 X + w_0$$

**잔차** - 실제 값과 회귀 모델의 차이에 따른 오류 값




- 최적의 회귀모델을 만든다는 것은 바로 전체 데이터의 잔차（오류 값） 합이 최소가 되는 모델을 만든다는 의미
- 동시에 오류 값 합이 최소가 될 수 있는 최적의 회귀 계수를 찾는다는 의미
- 오류 합을 계산할 때는 절댓값을 취해서 더하거나（Mean Absolute Error）, 오류 값의 제곱을 구해서 더하는 방식（RSS, Residual Sum of Square）을 취함
- RSS를 최소로 하는 회귀 계수를 학습을 통해서 찾는 것이 머신러닝 기반 회귀의 핵심 사항

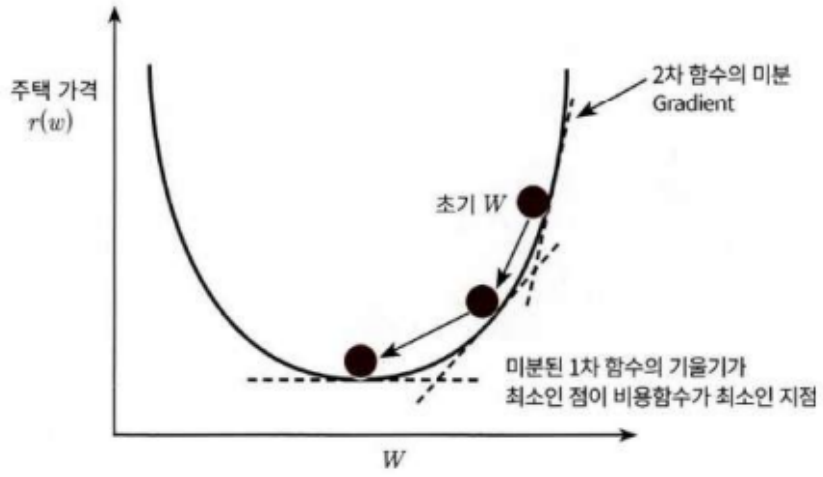
$$RSS(w_0, w_1) = \frac{1}{N}\sum_{i=1}^{N}(y_i - (w_0 + w_1 * x_i))^2$$

**($i$는 1부터 학습 데이터의 총 건수 $N$까지)**

**회귀 학습 = 비용 함수/손실 함수(RSS) 최소화 → 최적의 회귀 계수 도출**

## 03. 비용 최소화하기 - 경사 하강법（Gradient Descent）소개

- ‘데이터를 기반으로 알고리즘이 스스로 학습한다’는 머신러닝의 개념을 가능하게 만들어준 핵심 기법의 하나
- ‘점진적으로’ 반복적인 계산을 통해 w 파라미터 값을 업데이트하면서 오류 값이 최소가 되는 w 파라미터를 구하는 방식




**작동 원리**

1. 초기 w값을 설정
2. 비용 함수를 미분하여 기울기(Gradient) 계산
3. 기울기의 반대 방향으로 w 업데이트
4. 오류 값이 감소하도록 반복 (예: 100 → 90 → 80 → ...)
5. 오류가 충분히 작아지고 더 이상 유의미하게 감소하지 않으면 종료

핵심 원리: 비용 함수의 기울기가 0에 가까워지는 지점을 찾아 최소 비용을 만드는 최적의 w를 구함

비용함수 R(w)를 w1, w0으로 편미분한 결과

$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N}\sum_{i=1}^{N} -x_t * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N}\sum_{i=1}^{N} x_i * (\text{실제값}_i - \text{예측값}_i)$$

$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N}\sum_{i=1}^{N} -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N}\sum_{i=1}^{N} (\text{실제값}_i - \text{예측값}_i)$$

- Step 1: $w_1, w_0$를 임의의 값으로 설정하고 첫 비용 함수의 값을 계산합니다.
- Step 2: $w_1$을 $w_1 + \eta\frac{2}{N}\sum_{i=1}^{N} x_i * (\text{실제값}_i - \text{예측값}_i)$, $w_0$을 $w_0 + \eta\frac{2}{N}\sum_{i=1}^{N} (\text{실제값}_i - \text{예측값}_i)$으로 업데이트한 후 다시 비용 함수의 값을 계산합니다.
- Step 3: 비용 함수가 감소하는 방향성으로 주어진 횟수만큼 Step 2를 반복하면서 $w_1$과 $w_0$를 계속 업데이트합니다.

- 일반적으로 경사 하강법은 모든 학습 데이터에 대해 반복적으로 비용함수 최소화를 위한 값을 업데이트하기 때문에 수행 시간이 매우 오래 걸린다는 단점이 있음

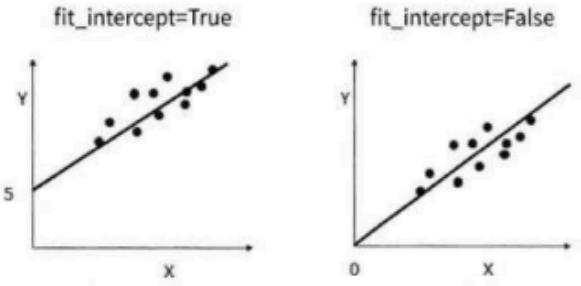
- 그 때문에 실전에서는 대부분 **확률적 경사 하강법(Stochastic Gradient Descent)**을 이용함
- **확률적 경사 하강법**은 전체 입력 데이터로 w가 업데이트되는 값을 계산하는 것이 아니라 일부 데이터만 이용해 w가 업데이트되는 값을 계산하므로 경사 하강법에 비해서 빠른 속도를 보장함

ipynb 코드 실습

## 04. 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

### LinearRegression 클래스 — Ordinary Least Squares

- LinearRegression 클래스는 예측값과 실제 값의 RSS(Residual Sum of Squares)를 최소화해
  OLS(Ordinary Least Squares) 추정 방식으로 구현한 클래스이다
- LinearRegression클래스는 fit() 메서드로 X, y 배열을 입력받으면 회귀 계수(Coefficients)인 W를 coef_속성에 저장함

| | |
|---|---|
| 입력 파라미터 | **fit_intercept**: 불린 값으로, 디폴트는 True입니다. Intercept(절편) 값을 계산할 것인지 말지를 지정합니다. 만일 False로 지정하면 intercept가 사용되지 않고 0으로 지정됩니다.<br>  <br>**normalize**: 불린 값으로 디폴트는 False입니다. fit_intercept가 False인 경우에는 이 파라미터가 무시됩니다. 만일 True이면 회귀를 수행하기 전에 입력 데이터 세트를 정규화합니다. |
| 속성 | **coef_**: fit() 메서드를 수행했을 때 회귀 계수가 배열 형태로 저장하는 속성. Shape는 (Target 값 개수, 피처 개수).<br>**intercept_**: intercept 값 |

**다중 공선성(Multicollinearity)** - 입력 피처(독립변수)간의 상관관계가 매우 높을 때 발생하는 문제

원인: 서로 비슷한 정보를 가진 피처들이 회귀 모델에 함께 포함되는 경우

문제점

- 회귀 계수의 분산이 커짐

- 작은 데이터 변화나 오류에도 회귀 계수가 민감하게 변함
- 회귀 계수의 안정성과 신뢰성이 떨어짐

해결 방법

1. 피처 제거: 상관관계가 높은 피처 중 중요한 피처만 남김
2. 규제(Regularization): 회귀 계수의 크기를 제한하여 모델을 안정화
3. PCA(주성분 분석): 다중 공선성이 있는 피처가 많을 경우 차원 축소 수행

**회귀 평가 지표**

실제 값과 회귀 예측값의 차이 값을 기반으로 한 지표가 중심

| 평가 지표 | 설명 | 수식 |
|---|---|---|
| MAE | Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다. | $MAE = \frac{1}{n}\sum_{i=1}^{n}\lvert Yi - \hat{Y}i\rvert$ |
| MSE | Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다. | $MSE = \frac{1}{n}\sum_{i=1}^{n}(Yi - \hat{Y}i)^2$ |
| RMSE | MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다. | $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Yi - \hat{Y}i)^2}$ |
| $R^2$ | 분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다. | $R^2 = \frac{\text{예측값 } Variance}{\text{실제값 } Variance}$ |

각 평가 방법에 대한 사이킷런의 API 및 cross_val_score나 GridSearchCV에서 평가 시 사용되는 scoring 파라미터의 적용 값

| 평가 방법 | 사이킷런 평가 지표 API | Scoring 함수 적용 값 |
|---|---|---|
| MAE | metrics.mean_absolute_error | 'neg_mean_absolute_error' |
| MSE | metrics.mean_squared_error | 'neg_mean_squared_error' |
| RMSE | metrics.mean_squared_error를 그대로 사용하되 squared 파라미터를 False로 설정. | 'neg_root_mean_squared_error' |
| MSLE | metrics.mean_squared_log_error | 'neg_mean_squared_log_error' |
| $R^2$ | metrics.r2_score | 'r2' |

- Scoring 함수의 scoring 파라미터 값 'neg_mean_absolute_error'가 의미하는 것은 -1 * metrics.
  mean_absolute_error()이다

**LinearRegression을 이용해 보스턴 주택 가격 회귀 구현**

ipynb 코드 실습

## 05. 다항 회귀와 과(대)적합/과소적합 이해

**다항 회귀 이해**

$$Y = W_1x + W_2x^2 + \ldots + W_nx^n + b$$

- 회귀가 독립변수의 단항식이 아닌 2차, 3차 방정식과 같은 다항식으로 표현되는 것
- 다항 회귀를 비선형 회귀로 혼동하기 쉽지만, 다항 회귀는 **선형 회귀**이다
- 사이킷런은 PolynomialFeatures 클래스를 통해 피처를 Polynomial(다항식) 피처로 변환함

ipynb 코드 실습

**다항 회귀를 이용한 과소적합 및 과적합 이해**

- 다항식의 차수가 높아질수록 매우 복잡한 피처 간의 관계까지 모델링이 가능함
- 하지만 다항 회귀의 **차수(degree)**를 높일수록 학습 데이터에만 너무 맞춘 학습이 이뤄져서 정작 테스트 데이터 환경에서는 오히려 예측 정확도가 떨어지는 **과적합**의 문제가 발생함

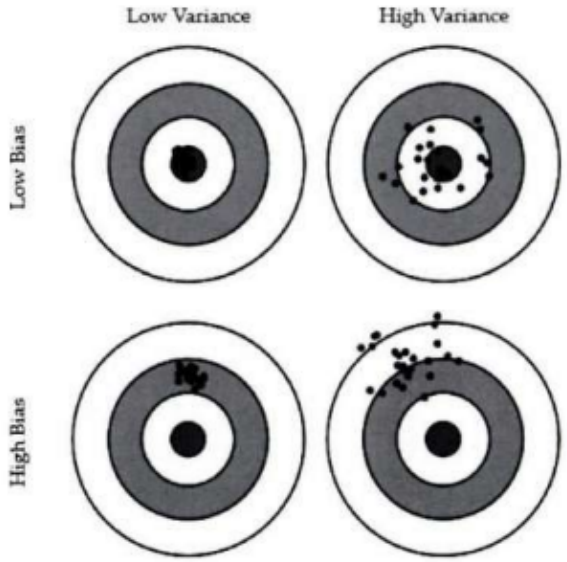
ipynb 코드 실습

**편향-분산 트레이드오프(Bias-Variance Trade off)**

편향(Bias)과 분산(Variance) 사이의 균형을 조절하여 모델의 예측 오류를 최소화하는 것

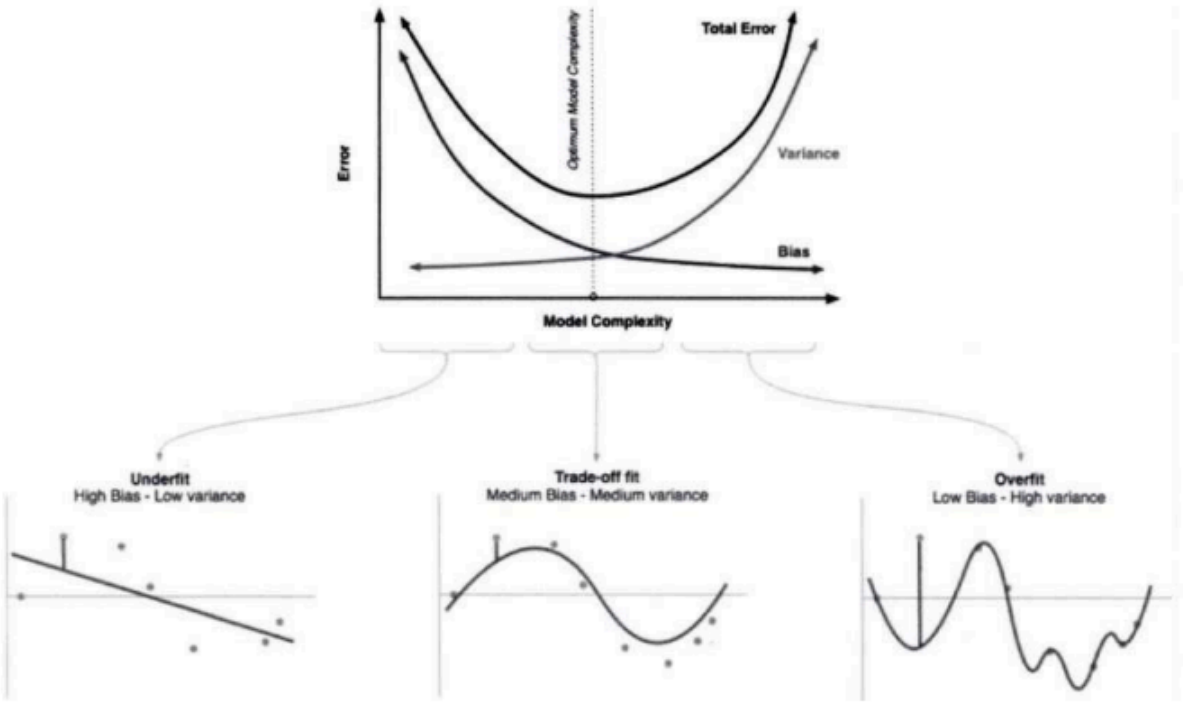
- 편향(Bias): 예측값이 실제값에서 얼마나 벗어나는지 나타내는 정도
  - 고편향(High Bias) → 모델이 지나치게 단순 → 과소적합(Underfitting)
- 분산(Variance): 학습 데이터 변화에 따라 모델의 예측이 얼마나 달라지는지 나타내는 정도

- 고분산(High Variance) → 모델이 지나치게 복잡 → 과적합(Overfitting)




〈 편향과 분산의 고/저에 따른 표현.
http://scott.fortmann-roe.com/docs/BiasVariance.html에서 발췌 〉

| 유형 | 특징 |
|---|---|
| 저편향 / 저분산 | 정확하고 안정적인 예측 (이상적) |
| 저편향 / 고분산 | 평균적으로 정확하지만 예측 변동이 큼 |
| 고편향 / 저분산 | 예측이 일정하지만 실제값에서 벗어남 |
| 고편향 / 고분산 | 예측이 부정확하고 변동도 큼 |




〈 편향과 분산에 따른 전체 오류 값(Total Error) 곡선. http://scott.fortmann-roe.com/docs/BiasVariance.html에서 발췌. 〉

- 골디락스(Goldilocks) 지점: 편향과 분산이 적절한 균형을 이루어 전체 예측 오류가 최소가 되는 지점

## 06. 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

### 규제 선형 모델의 개요

- 회귀 모델은 적절히 데이터에 적합하면서도 회귀 계수가 기하급수적으로 커지는 것을 제어할 수 있어
  야 한다
- 비용 함수는 학습 데이터의 잔차 오류 값을 최소로 하는 RSS 최소화 방법과 과적합을 방지하기 위해
  회귀 계수 값이 커지지 않도록 하는 방법이 서로 균형을 이뤄야 함

$$\text{비용 함수 목표} = \text{Min}(\text{RSS(W)} + \text{alpha} * \| W \|_2^2)$$

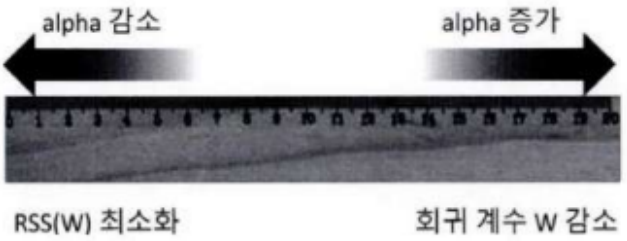
### 규제(Regularization)

비용 함수에 페널티를 부여하여 회귀 계수 \(W\)의 크기를 줄이고 과적합(Overfitting)을 방지하는 방법

α(Alpha): 규제의 강도를 조절하는 하이퍼파라미터

- α ↑ → 규제 강화 → 회귀 계수 W 감소 → 과적합 완화
- α ↓ → 규제 약화 → 회귀 계수 W에 대한 제약 감소 → 학습 데이터 적합도 향상
- α = 0 → 규제 적용 X

- alpha = 0인 경우는 W가 커도 alpha * $\|W\|_2^2$가 0이 되어 비용 함수는 Min(RSS(W))
- alpha = 무한대인 경우 alpha * $\|W\|_2^2$도 무한대가 되므로 비용 함수는 W를 0에 가깝게 최소화 해야 함.



〈alpha 튜닝 파라미터를 통한 RSS 최소화와 회귀 계수 크기 감소의 균형 조정〉

### 규제의 종류

| 구분 | L2 규제 (Ridge) | L1 규제 (Lasso) |
|---|---|---|
| 페널티 | 회귀 계수 W의 제곱 | 회귀 계수 W의 절댓값 |
| 특징 | 회귀 계수의 크기를 줄임 | 일부 회귀 계수를 0으로 만듦 |
| 효과 | 과적합 완화, 모델 안정화 | 과적합 완화, 불필요한 피처 제거 효과 |

### 릿지 회귀

사이킷런은 Ridge 클래스를 통해 릿지 회귀를 구현함

ipynb 코드 실습

### 라쏘 회귀

L1 규제는 적절한 피처만 회귀에 포함시키는 피처 선택의 특성을 가지고 있다

사이킷런은 Lasso 클래스를 통해 라쏘 회귀를 구현함

ipynb 코드 실습

### 엘라스틱넷 회귀

- L2 규제와 L1 규제를 결합한 회귀
- 목표: 식을 최소화하는 W를 찾는 것

$$\text{RSS}(W) + \text{alpha2} * \|W\|_2 + \text{alpha1} * \|W\|_1$$

- 특징
  - L1 규제 → 중요하지 않은 피처의 회귀 계수를 0으로 만듦
  - L2 규제 → 회귀 계수의 크기를 줄여 안정성 향상
  - 상관관계가 높은 피처들 중 일부만 선택하는 라쏘의 단점을 보완
- 단점: L1과 L2를 함께 적용하므로 수행 시간이 상대적으로 오래 걸릴 수 있음

| 파라미터 | 의미 |
| --- | --- |
| `alpha` | 전체 규제 강도 (a+b) |
| `l1_ratio` | L1 규제의 비율 (a/(a+b)) |

**규제 식: a * L1 + b * L2**

- **a**: L1 규제 강도
- **b**: L2 규제 강도
- `l1_ratio = 0` → L2 규제만 적용 (Ridge 형태)
- `l1_ratio = 1` → L1 규제만 적용 (Lasso와 동일)
- `0 < l1_ratio < 1` → L1과 L2 규제 혼합

ipynb 코드 실습

**선형 회귀 모델을 위한 데이터 변환**

- 선형 회귀 모델과 같은 선형 모델은 일반적으로 피처와 타깃값 간에 선형의 관계가 있다고 가정하고,
  이러한 최적의 선형함수를 찾아내 결괏값을 예측함
- 선형 회귀 모델은 피처값과 타깃값의 분포가 정규 분포(즉 평균을 중심으로 종 모양으로 데이터 값이 분포된 형태) 형태를 매우 선호함
- 따라서 선형 회귀 모델을 적용하기 전에 먼저 데이터에 대한 스케일링/정규화 작업을 수행하는 것이 일반적임
- 스케일링/정규화 작업을 선행한다고 해서 무조건 예측 성능이 향상되는 것은 아니며, 일반적으로 중요 피처들이나 타깃값의 분포도가 심하게 왜곡됐을 경우에 이러한 변환 작업을 수행함

1. StandardScaler 클래스를 이용해 평균이 0, 분산이 1인 표준 정규 분포를 가진 데이터 세트로 변환하거나 MinMaxScaler 클래스를 이용해 최솟값이 0이고 최댓값이 1인 값으로 정규화를 수행합니다.
2. 스케일링/정규화를 수행한 데이터 세트에 다시 다항 특성을 적용하여 변환하는 방법입니다. 보통 1번 방법을 통해 예측 성능에 향상이 없을 경우 이와 같은 방법을 적용합니다.
3. 원래 값에 log 함수를 적용하면 보다 정규 분포에 가까운 형태로 값이 분포됩니다. 이러한 변환을 로그 변환(Log Transformation)이라고 부릅니다. 로그 변환은 매우 유용한 변환이며, 실제로 선형 회귀에서는 앞에서 소개한 1, 2번 방법보다 로그 변환이 훨씬 많이 사용되는 변환 방법입니다. 왜냐하면 1번 방법의 경우 예측 성능 향상을 크게 기대하기 어려운 경우가 많으며 2번 방법의 경우 피처의 개수가 매우 많을 경우에는 다항 변환으로 생성되는 피처의 개수가 기하급수로 늘어나서 과적합의 이슈가 발생할 수 있기 때문입니다.

ipynb 코드 실습

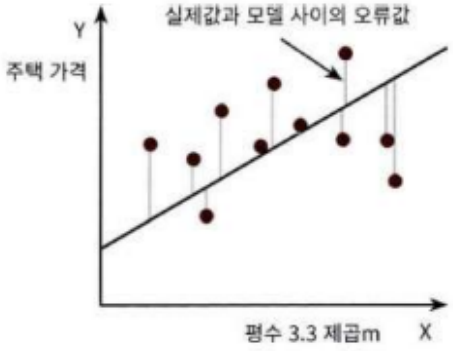
## 07. 로지스틱 회귀

### 시그모이드 함수(Sigmoid Function)

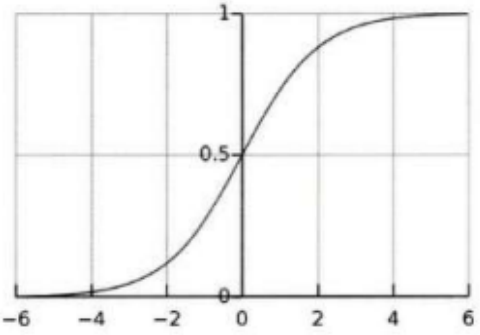
$$f(x) = \frac{1}{1 + e^{-x}}$$

- 입력값 x를 항상 0과 1 사이의 값으로 변환하는 S자 형태의 함수
- 특징
  - x ↑ → 출력값이 1에 가까워짐
  - x ↓ → 출력값이 0에 가까워짐
  - x=0 → 출력값 = 0.5
- 역할: 출력값을 확률(0~1)로 표현하여 이진 분류에 활용

### 로지스틱 회귀(Logistic Regression)

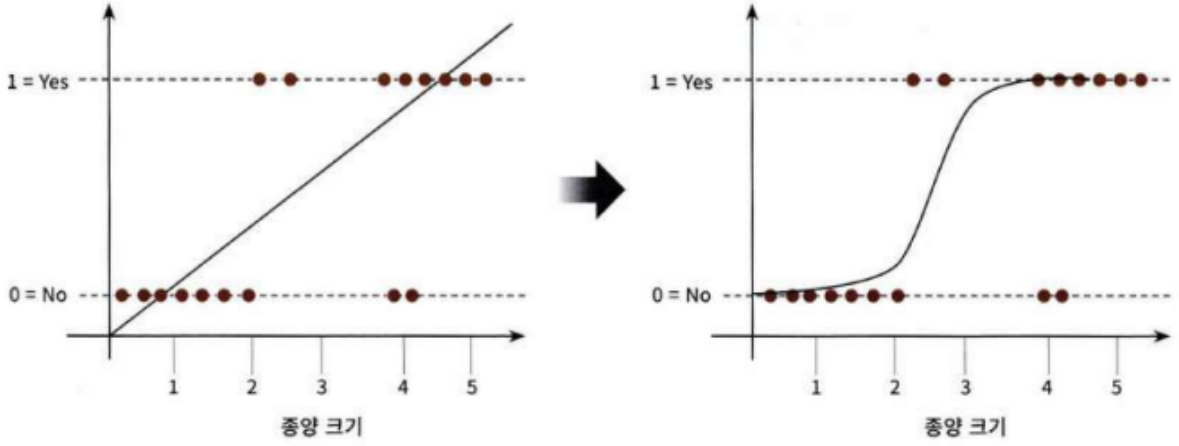


〈 선형 회귀의 선형 함수 〉



〈 로지스틱 회귀의 시그모이드 함수 〉

- 선형 회귀 방식을 기반으로 시그모이드 함수를 적용하여 분류(Classification) 를 수행하는 알고리즘
- 특징
  - 선형 회귀 → 연속형 값 예측 (예: 부동산 가격)
  - 로지스틱 회귀 → 특정 클래스에 속할 확률 예측 (예: 악성 종양 여부)
- 작동 원리
  1. 입력 데이터를 선형식으로 계산
  2. 시그모이드 함수로 0~1 사이의 확률로 변환
  3. 기준값(일반적으로 0.5)에 따라 0 또는 1로 분류




사이킷런은 로지스틱 회귀를 위해서 LogisticRegression 클래스를 제공함

ipynb 코드 실습

## 08. 회귀 트리

- 회귀 함수를 이용하는 대신 결정 트리(Decision Tree) 구조를 이용하여 연속형 값을 예측하는 회귀 방식
- 특징
  - 회귀 트리(Regression Tree)를 생성하여 예측 수행
  - 분류 트리와 기본 구조는 동일
  - 리프 노드(Leaf Node) 에서 예측값을 결정하는 방식에 차이가 있음

**분류 트리 vs 회귀 트리**

| 구분 | 분류 트리 | 회귀 트리 |
| --- | --- | --- |
| 목적 | 범주형 값 예측 | 연속형 값 예측 |
| 예측 결과 | 클래스 레이블 | 수치형 값 |
| 리프 노드 | 특정 클래스 결정 | 해당 노드 데이터의 평균값 계산 |
| 예시 | 합격 / 불합격 | 집값, 매출액 예측 |

트리 기반 회귀 = 결정 트리로 데이터 분할 → 리프 노드에 속한 데이터의 평균값을 최종 예측값으로 사용

**사이킷런의 트리 기반 회귀와 분류의 Estimator 클래스**

| 알고리즘 | 회귀 Estimator 클래스 | 분류 Estimator 클래스 |
| --- | --- | --- |
| Decision Tree | DecisionTreeRegressor | DecisionTreeClassifier |
| Gradient Boosting | GradientBoostingRegressor | GradientBoostingClassifier |
| XGBoost | XGBRegressor | XGBClassifier |
| LightGBM | LGBMRegressor | LGBMClassifier |